

## Lecture 6

# Linear Regression (2)

Regularized Least Squares and Illconditioning

**Keji He @ SDU-AI**

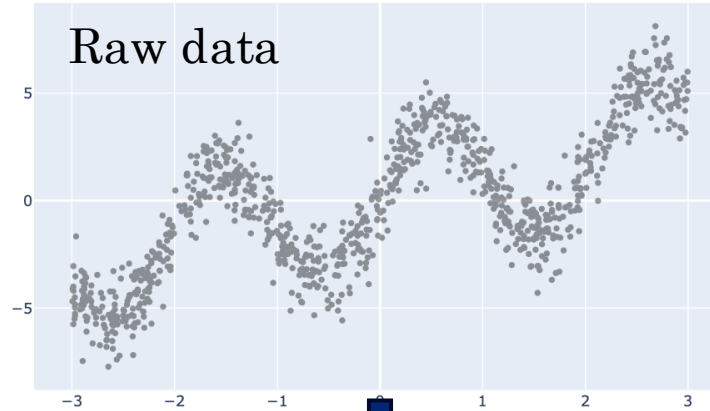
# Roadmap

- Regularized Least Squares
- When Normal Equation Gets Tricky

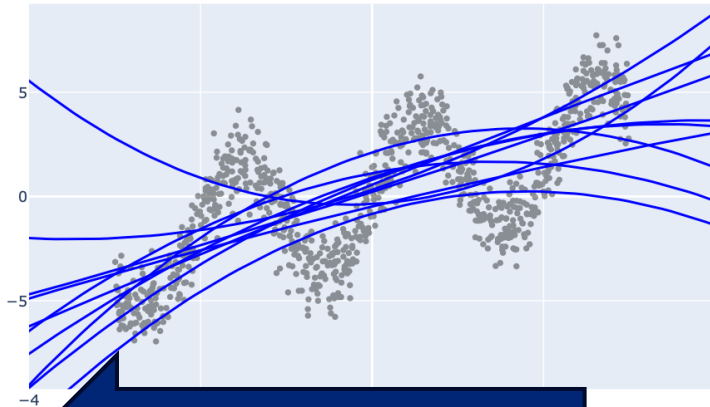
# Regularized Least Squares

- Regularized Least Squares
- When Normal Equation Gets Tricky

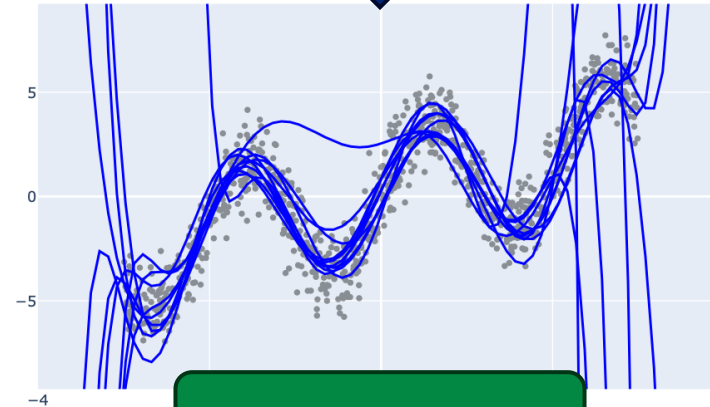
# Complexity and Overfitting



Fit models to  
samples of data

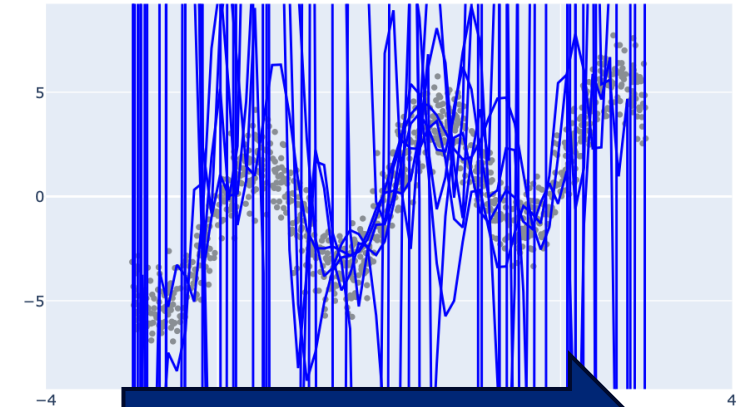


Underfitting



"Sweet Spot"

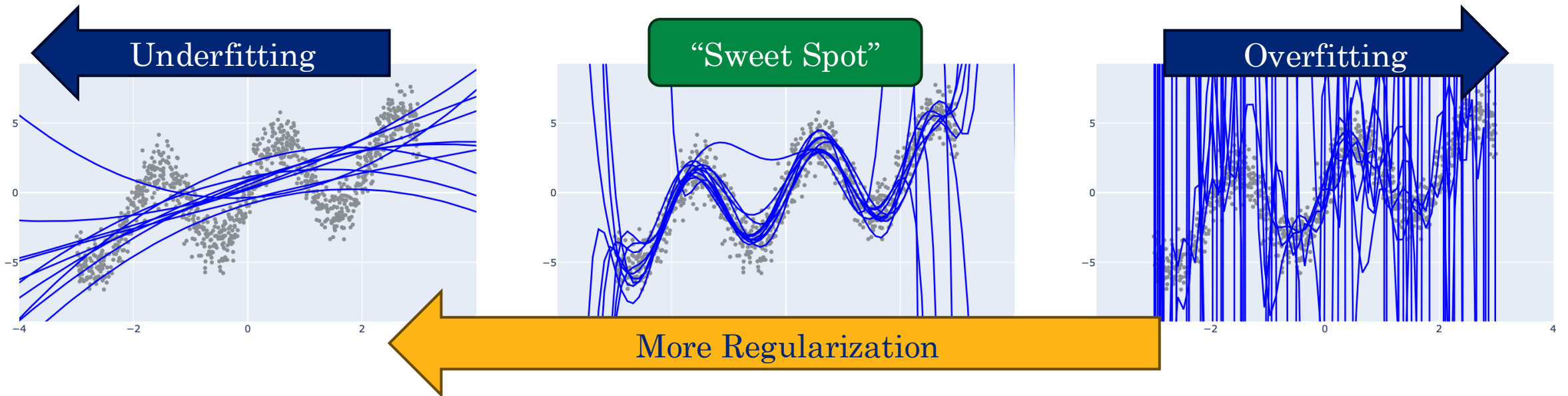
More complex isn't  
always better.



Overfitting

# Regularization

Regularization is the process of adding **constraints** or **penalties** to the learning process to **improve generalization**.



Many **models** and **learning algorithms** have methods to **tune** the **regularization** during the training process.

# Regularization

**Regularization** restricts complexity by limiting the magnitudes of the model parameters  $w$ .

We capture this restriction through the function  $\text{Reg}[w]$  and restricting its value to be less than  $C$ .

Our Error function becomes a constrained function, so we minimize:

$$\text{Error}[h_w; \mathcal{D}_{\text{training}}] \quad \text{such that} \quad \text{Reg}[w] < C$$

# Regularization – Lagrangian Relaxation

Our Error function becomes a constrained function, so we minimize:

$$\text{Error}[h_w; \mathcal{D}_{\text{training}}] \quad \text{such that} \quad \text{Reg}[w] \leq C$$

This constraint complicates our optimization.

What if we assign a penalty for each violation of the constraint?

## Idea of Lagrangian

Introduce a penalty for breaking the constraint.

$$\underbrace{\lambda}_{\text{Imposed penalty}} \underbrace{(\text{Reg}[w] - C)}_{\text{Violation magnitude}}$$

The updated Error function

$$E(w) = \text{Error}[h_w; \mathcal{D}_{\text{training}}] + \lambda \text{Reg}[w]$$

**Note:** we did not include  $-\lambda C$  since it won't impact the optimization of  $w$ .

# From **Reg**[*w*] → Norms

- Generally, **norms** are the standard choices for **Reg**[*w*].
- Norm definition:  $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$
- What is a norm?
  - **Positive Definiteness** (正定性):
$$\|v\| \geq 0, \|v\| = 0 \iff \vec{v} = \vec{0}$$
  - **Positive homogeneity** (正齐次性):  $\|\alpha v\| = |\alpha| \|v\|$
  - **Triangle inequality** (三角不等式):  $\|u + v\| \leq \|u\| + \|v\|$

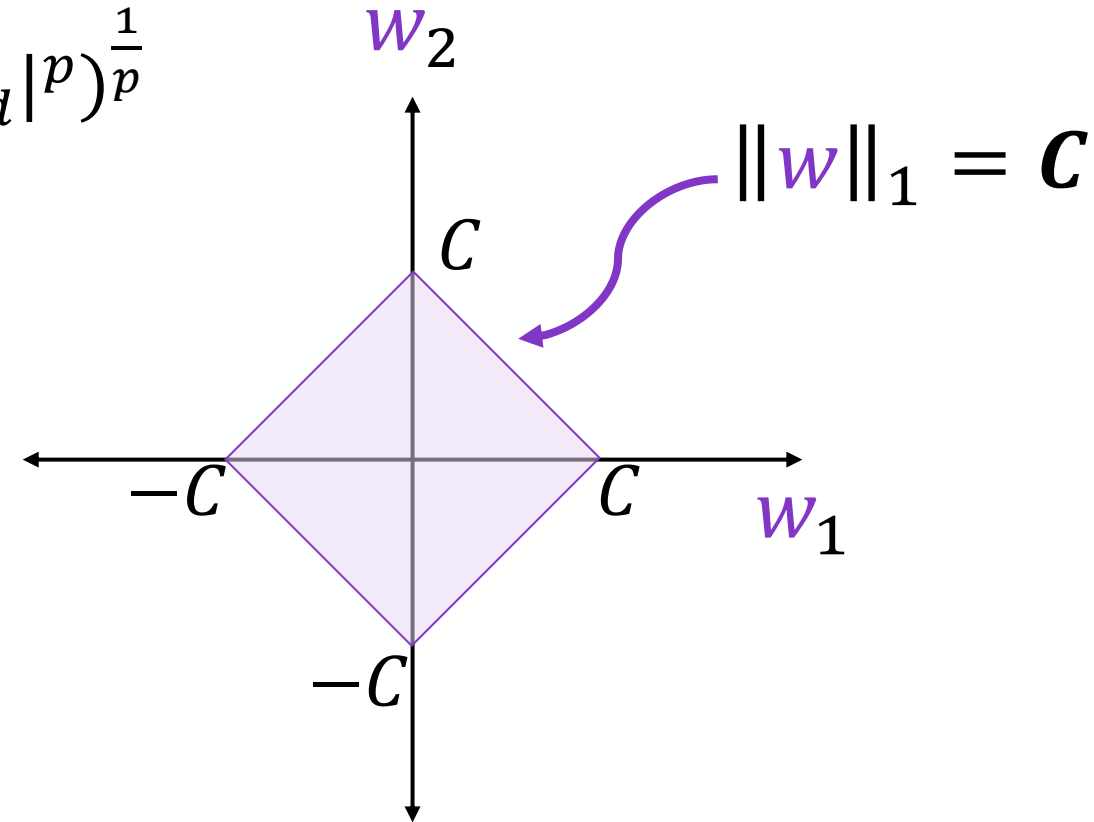


# From **Reg**[*w*] → Norms

- Norm definition:  $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$
- Example of norms:
  - $L_1(w): \|w\|_1 = \sum_d |w_d|$
  - $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$
  - $L_\infty(w): \|w\|_\infty = \max_d |w_d|$

# Reg[ $w$ ] $\rightarrow$ Norms $\rightarrow L_1$ Regularization

- Norm definition:  $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$
- Example of norms:
  - $L_1(w): \|w\|_1 = \sum_d |w_d|$
  - $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$
  - $L_\infty(w): \|w\|_\infty = \max_d |w_d|$



# Reg[ $w$ ] $\rightarrow$ Norms $\rightarrow L_2$ Regularization

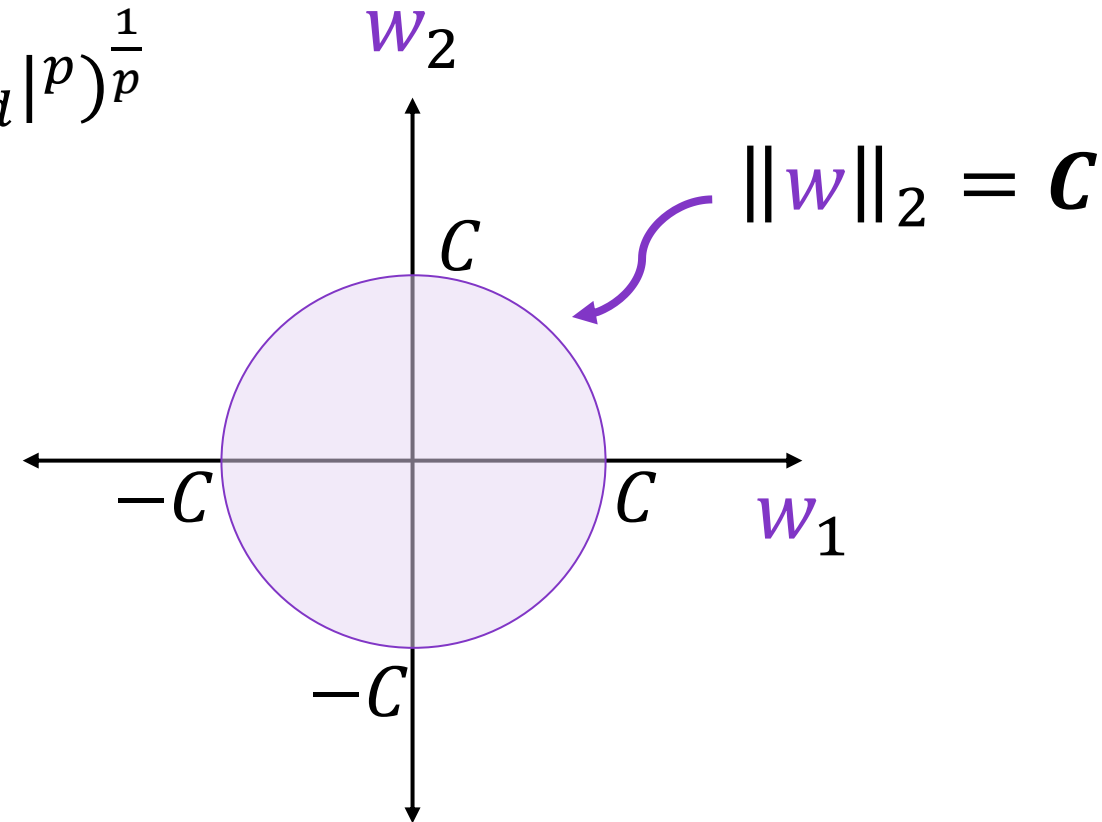
- Norm definition:  $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$

- Example of norms:

- $L_1(w): \|w\|_1 = \sum_d |w_d|$

- $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$

- $L_\infty(w): \|w\|_\infty = \max_d |w_d|$



# Reg[w] → Norms → $L_\infty$ Regularization

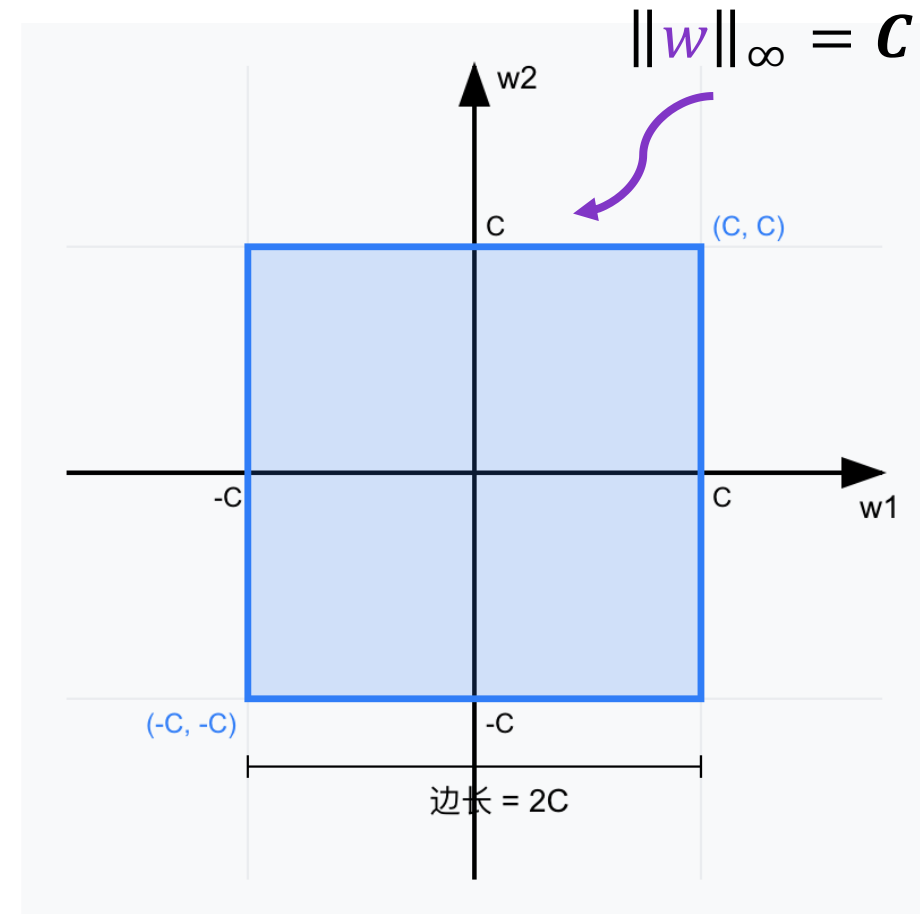
- Norm definition:  $L_p(w): \|w\|_p = (\sum_d |w_d|^p)^{\frac{1}{p}}$

- Example of norms:

- $L_1(w): \|w\|_1 = \sum_d |w_d|$

- $L_2(w): \|w\|_2 = \sqrt{\sum_d w_d^2}$

- $L_\infty(w): \|w\|_\infty = \max_d |w_d|$



# Regularization

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w \left( \underbrace{E_D(w)}_{\text{Data-dependent Error}} + \lambda \text{Reg}[w] \right)$$

Data-dependent  
Error

# Regularization

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

$$w^* = \arg \min_w \left( \underbrace{E_D(w)}_{\text{Data-dependent Error}} + \lambda \underbrace{E_w[w]}_{\text{Regularization Function}} \right)$$

# Regularization

We typically use **iterative optimization algorithms** to train (fit) the model by solving the following problem:

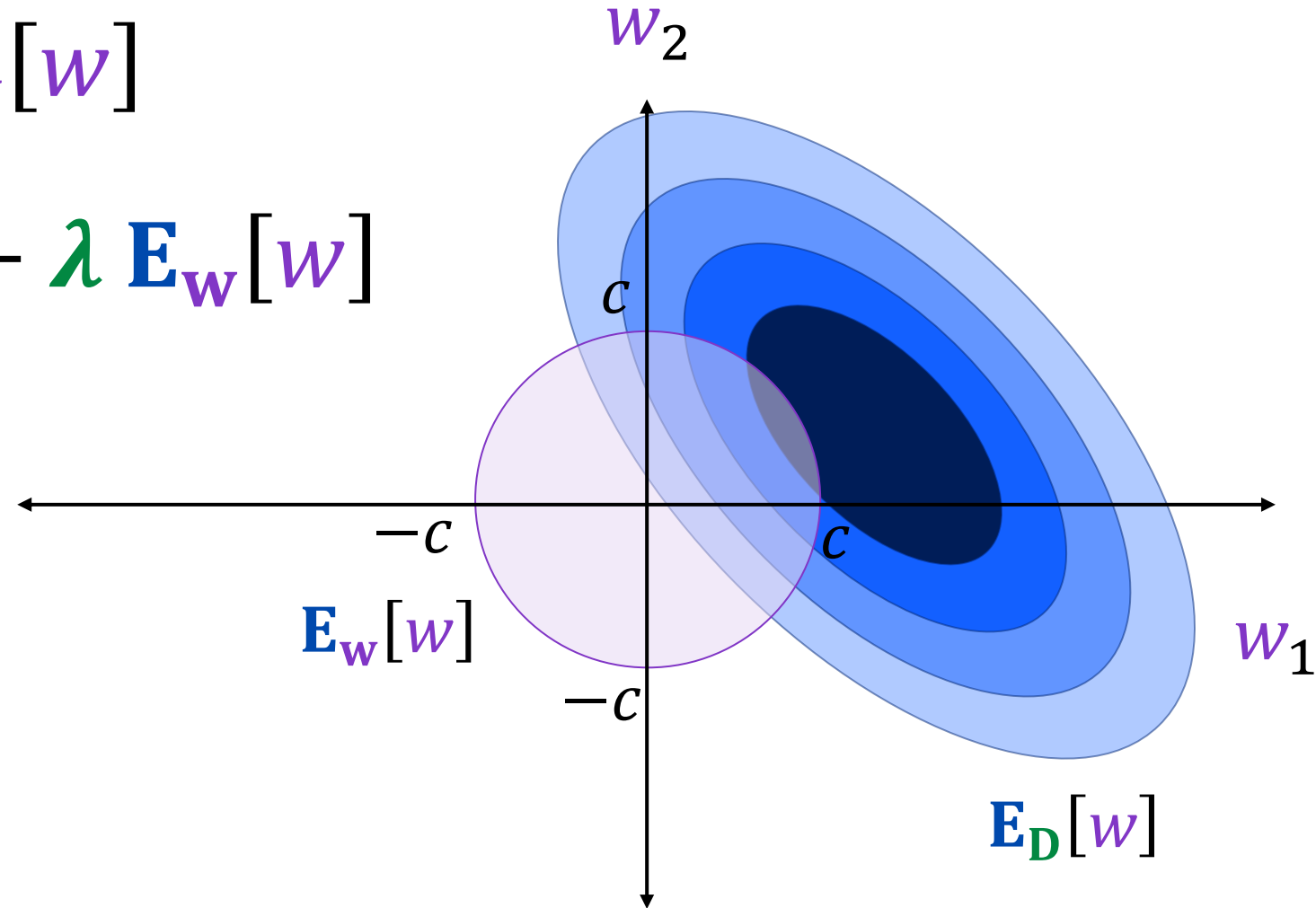
$$w^* = \arg \min_w (\underbrace{E_D(w)}_{\text{Data-dependent Error}} + \underbrace{\lambda}_{\text{Regularizer Hyperparameter}} \underbrace{E_w[w]}_{\text{Regularization Function}})$$

The diagram illustrates the components of the regularization equation. The equation is  $w^* = \arg \min_w (E_D(w) + \lambda E_w[w])$ . Brackets and arrows identify the parts:  $E_D(w)$  is the Data-dependent Error,  $\lambda$  is the Regularizer Hyperparameter, and  $E_w[w]$  is the Regularization Function.

# Linear Regression with L2 Regularization (Ridge)

$$E(w) = E_D[w] + \lambda E_w[w]$$

$$w^* = \arg \min_w E_D[w] + \lambda E_w[w]$$





# L2 Regularization (Ridge)

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_{\mathbf{D}}[\mathbf{w}] + \lambda \mathbf{E}_{\mathbf{w}}[\mathbf{w}]}_{\text{L2 Regularization (Ridge)}}$$

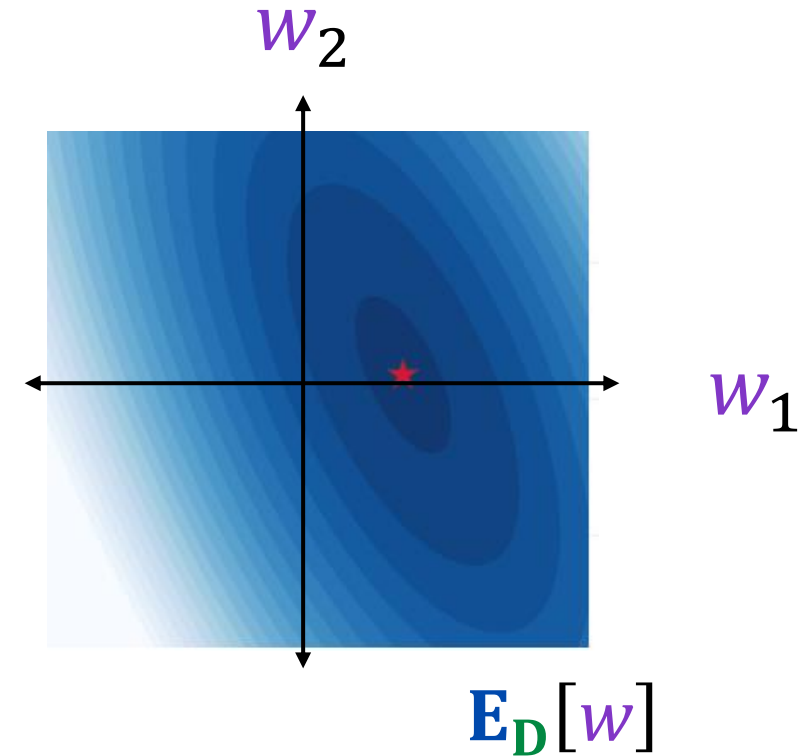
$$\frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

$$\frac{1}{2} (\|\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

$$\frac{1}{2} (\|e_n\|_2)^2$$

length of the residual vector

$$L_2[\mathbf{w}] = \sqrt{\sum_d \mathbf{w}_d^2}$$



# L2 Regularization (Ridge)

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_{\mathbf{D}}[\mathbf{w}]} + \underbrace{\lambda \mathbf{E}_{\mathbf{w}}[\mathbf{w}]}$$

$$\frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

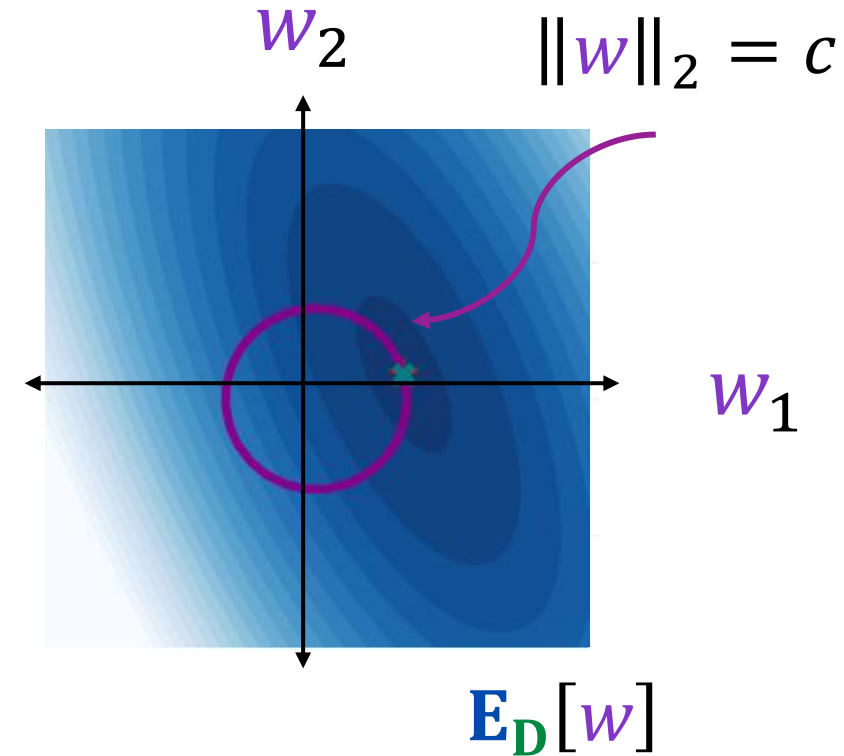
$$\frac{1}{2} L_2[\mathbf{w}]^2 = \frac{1}{2} \sum_d \mathbf{w}_d^2$$

$$\frac{1}{2} (\|\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

$$\frac{1}{2} (\|e_n\|_2)^2$$

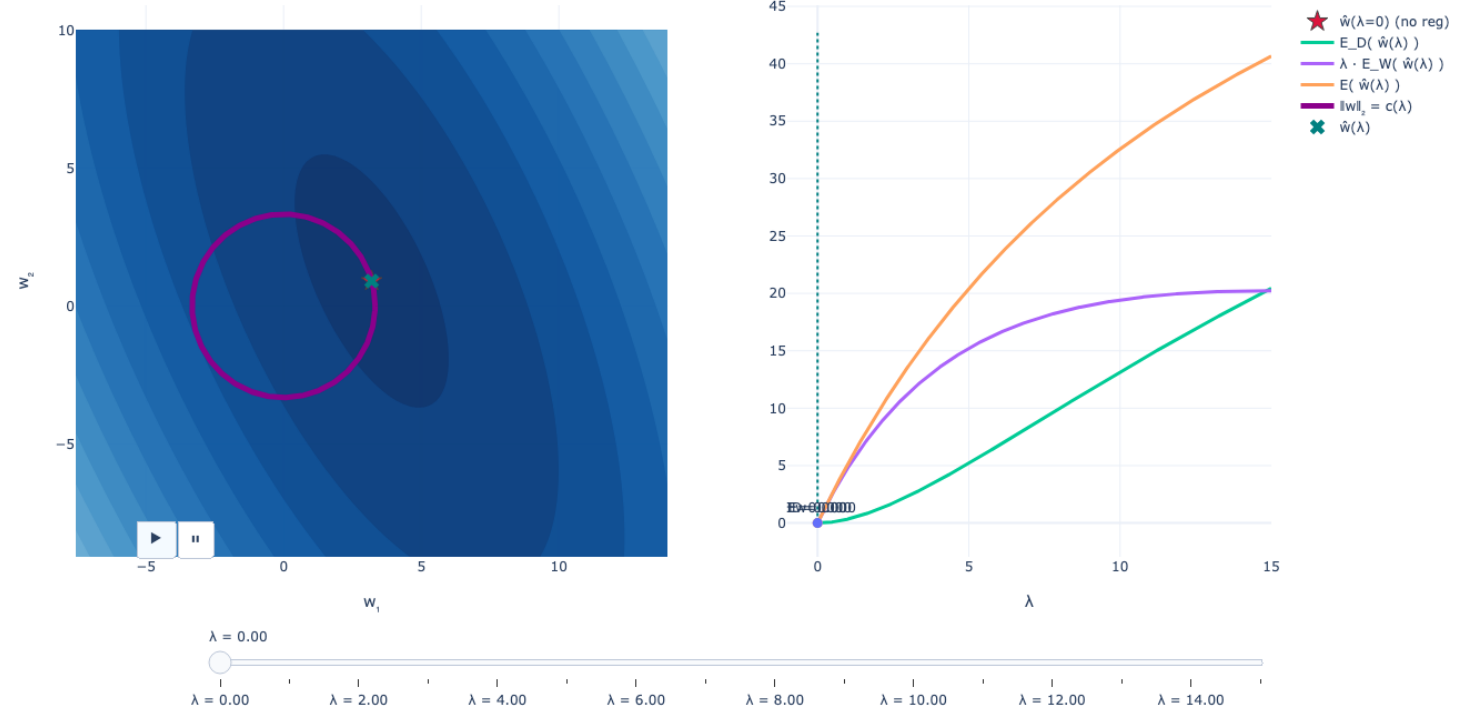
length of the residual vector

$$L_2[\mathbf{w}] = \sqrt{\sum_d \mathbf{w}_d^2}$$



# L2 Regularization Demo

[Interactive link](#)



# L2 Regularization (Ridge)

## 1. 定义目标函数 (Loss Function)

目标函数由 残差平方和 (RSS) 项和 L2 正则化 项组成：

$$J(w) = \frac{1}{2} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

利用向量内积的性质  $\|a\|_2^2 = a^T a$ ，展开目标函数：

$$J(w) = \frac{1}{2} (Xw - y)^T (Xw - y) + \frac{\lambda}{2} w^T w$$

$$J(w) = \frac{1}{2} (w^T X^T X w - w^T X^T y - y^T X w + y^T y) + \frac{\lambda}{2} w^T w$$

由于  $w^T X^T y$  是一个标量，其转置等于自身（即  $w^T X^T y = y^T X w$ ），公式可简化为：

$$J(w) = \frac{1}{2} (w^T X^T X w - 2y^T X w + y^T y) + \frac{\lambda}{2} w^T w$$

# L2 Regularization (Ridge)

## 2. 计算梯度 (Gradient)

我们需要对向量  $w$  求偏导。利用矩阵微分公式：

- $\frac{\partial(b^T w)}{\partial w} = b$
- $\frac{\partial(w^T A w)}{\partial w} = (A + A^T)w$  (若  $A$  是对称阵, 则为  $2Aw$ )

对  $J(w)$  求导：

$$\nabla_w J(w) = \frac{1}{2}(2X^T X w - 2X^T y) + \frac{\lambda}{2}(2w)$$

$$\nabla_w J(w) = X^T X w - X^T y + \lambda w$$

## 3. 求解驻点 (Stationary Point)

令梯度等于零向量  $\mathbf{0}$ ：

$$X^T X w + \lambda w = X^T y$$

提取公因子  $w$ ：

$$(X^T X + \lambda I)w = X^T y$$

其中  $I$  是  $(D + 1) \times (D + 1)$  的单位矩阵。

---

## 4. 最终解析解

只要  $\lambda > 0$ , 矩阵  $(X^T X + \lambda I)$  一定是正定且可逆的 (即使  $X^T X$  本身不满秩)。因此, 左乘逆矩阵得到：

$$w^* = (X^T X + \lambda I)^{-1} X^T y$$

# L1 Regularization (Lasso)

$$L_1[\mathbf{w}] = \sum_j |\mathbf{w}_j|$$

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \underbrace{\mathbf{E}_D[\mathbf{w}]} + \underbrace{\lambda \mathbf{E}_w[\mathbf{w}]}$$

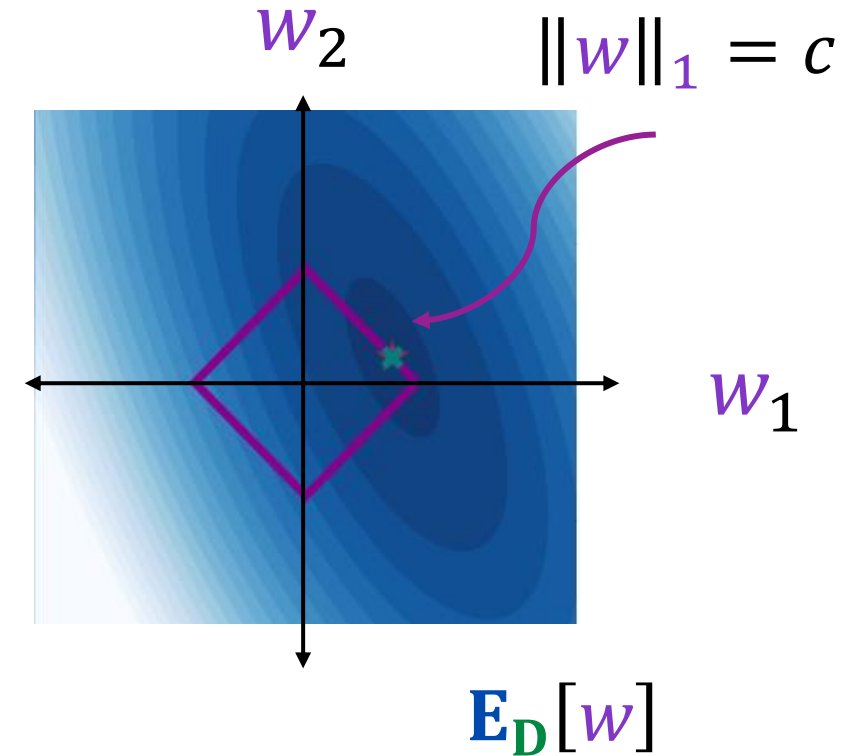
$$\frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

$$\frac{1}{2} L_1[\mathbf{w}] = \frac{1}{2} \sum_d |\mathbf{w}_d|$$

$$\frac{1}{2} (\|\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w})\|_2)^2$$

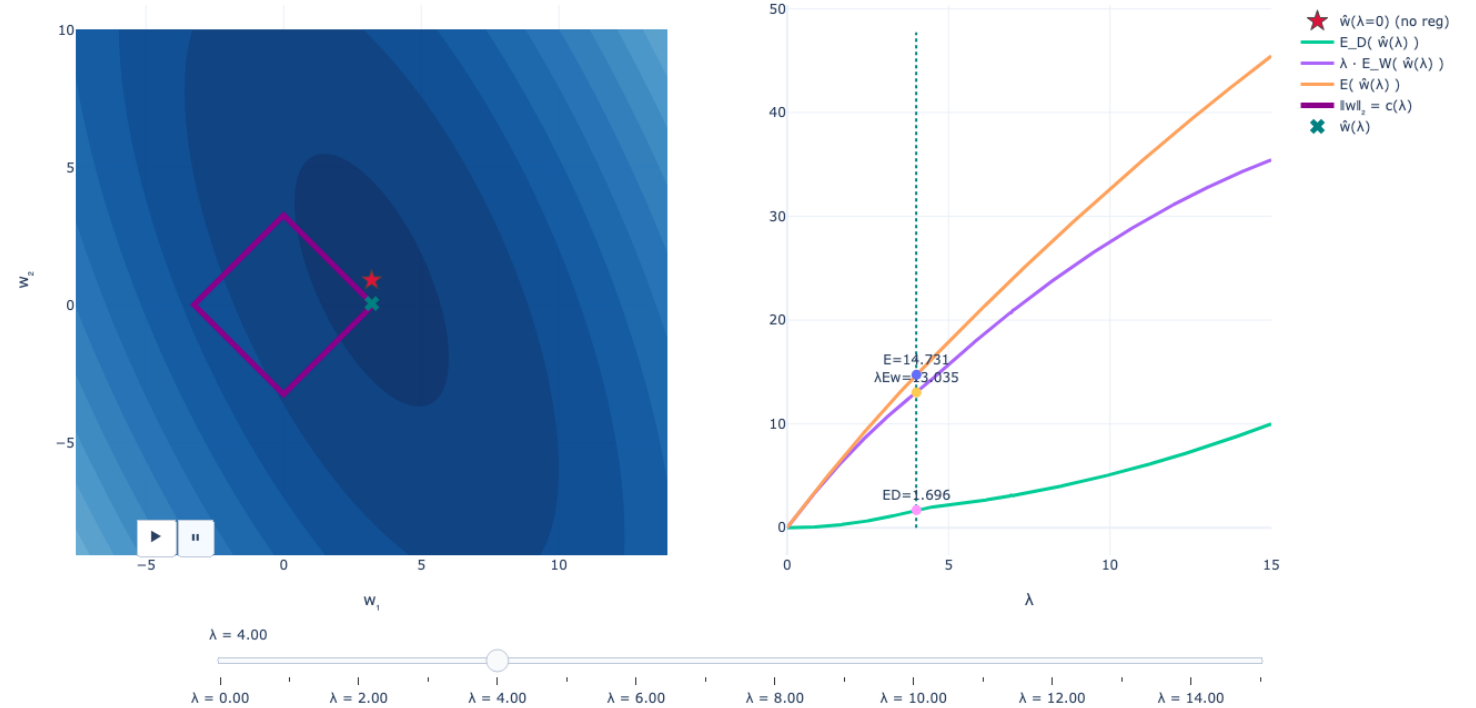
$$\frac{1}{2} (\|e_n\|_2)^2$$

length of the residual vector



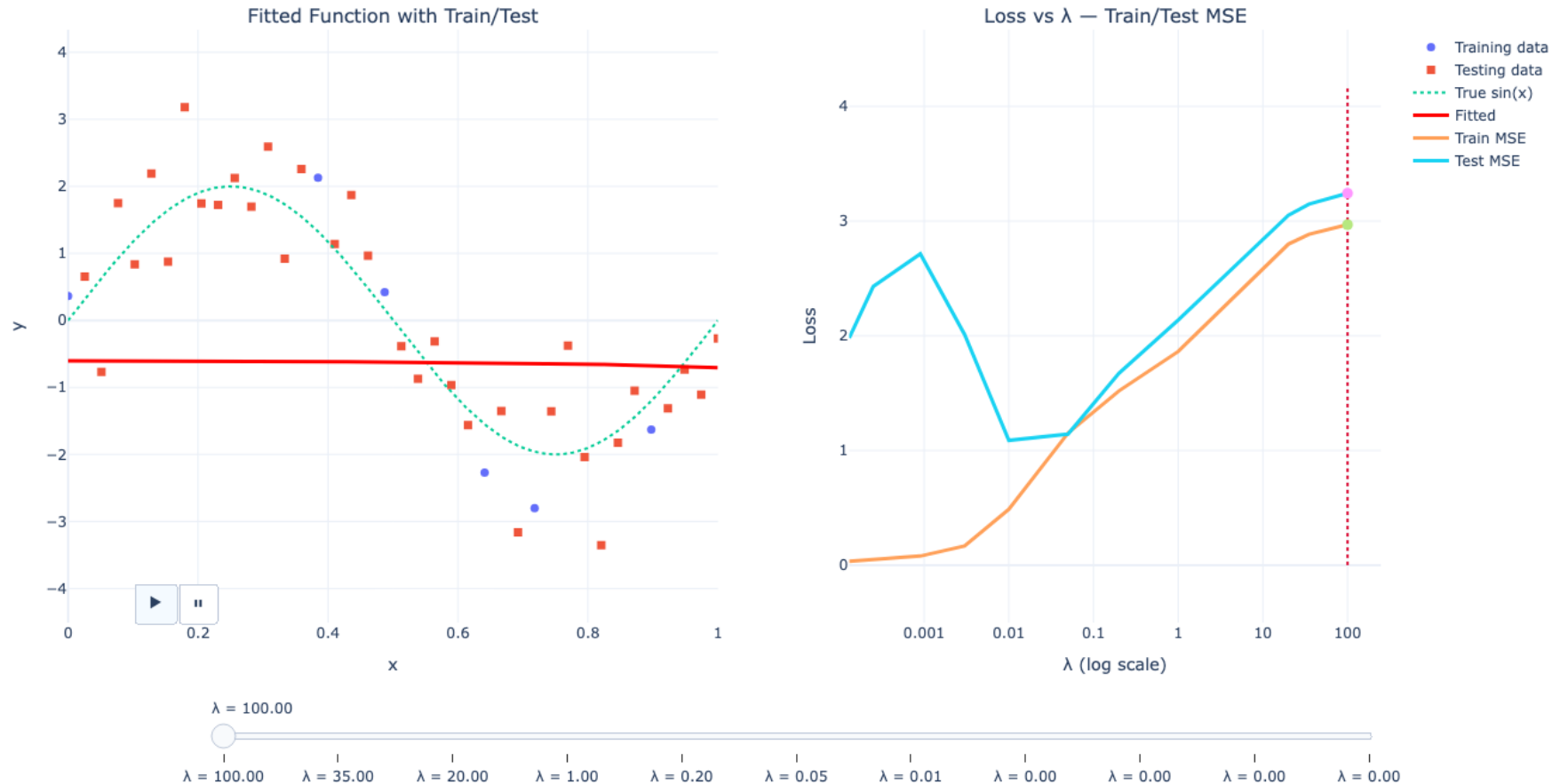
# L2 Regularization Demo

[Interactive link](#)



# Impact of Regularization - Ridge

- $\lambda$  controls complexity and helps avoid overfitting.



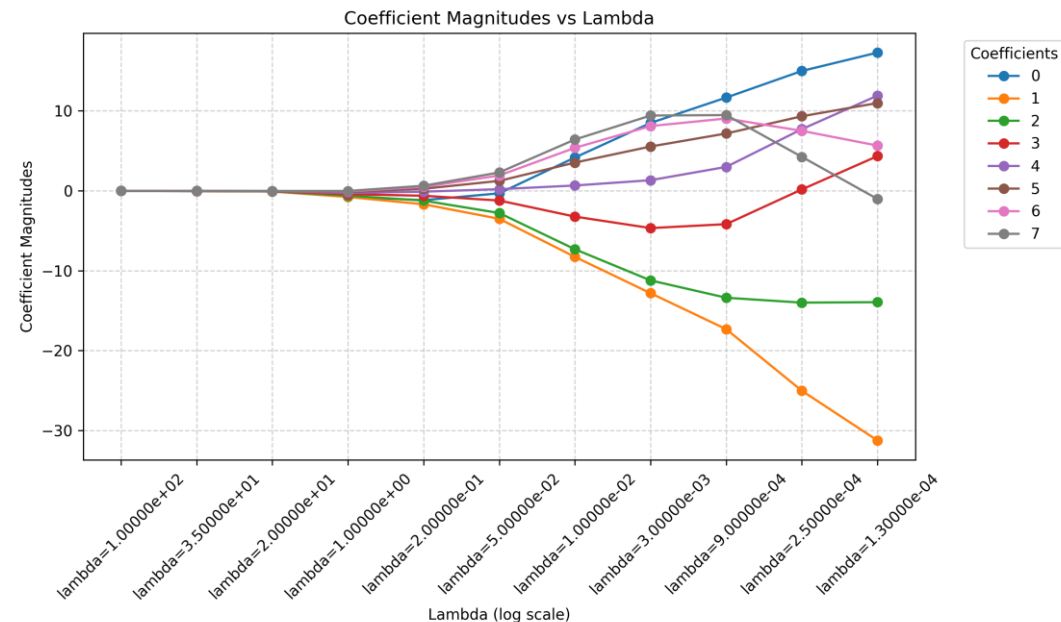
[Interactive link](#)



# Impact of Regularization - Ridge

- $\lambda$  controls complexity and helps avoid overfitting.

|                   | lambda=1.00000e+02 | lambda=3.50000e+01 | lambda=2.00000e+01 | lambda=1.00000e+00 | lambda=2.00000e-01 | lambda=5.00000e-02 | lambda=1.00000e-02 | lambda=3.00000e-03 | lambda=9.00000e-04 | lambda=2.50000e-04 | lambda=1.30000e-04 |
|-------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|--------------------|
| Polynomial Degree |                    |                    |                    |                    |                    |                    |                    |                    |                    |                    |                    |
| 0                 | -0.018966          | -0.052435          | -0.088490          | -0.750335          | -1.178313          | -0.318366          | 4.162396           | 8.485826           | 11.662871          | 14.992023          | 17.289582          |
| 1                 | -0.019036          | -0.052632          | -0.088828          | -0.777657          | -1.686689          | -3.508704          | -8.271798          | -12.817502         | -17.319566         | -25.023898         | -31.248832         |
| 2                 | -0.016455          | -0.045379          | -0.076367          | -0.611629          | -1.230065          | -2.784778          | -7.316610          | -11.216560         | -13.384204         | -13.996051         | -13.938533         |
| 3                 | -0.013521          | -0.037154          | -0.062269          | -0.426514          | -0.623082          | -1.219114          | -3.232832          | -4.667422          | -4.177037          | 0.170150           | 4.339540           |
| 4                 | -0.010943          | -0.029948          | -0.049953          | -0.273166          | -0.113433          | 0.200421           | 0.657859           | 1.322626           | 2.988265           | 7.711474           | 11.909930          |
| 5                 | -0.008863          | -0.024148          | -0.040075          | -0.159327          | 0.251802           | 1.238814           | 3.527585           | 5.538996           | 7.166978           | 9.322972           | 10.980703          |
| 6                 | -0.007234          | -0.019625          | -0.032402          | -0.079540          | 0.488914           | 1.910190           | 5.373464           | 8.096688           | 9.045151           | 7.495621           | 5.653301           |
| 7                 | -0.005969          | -0.016127          | -0.026494          | -0.025775          | 0.628286           | 2.293662           | 6.412034           | 9.413875           | 9.471402           | 4.241110           | -1.029883          |



# When Normal Equation Gets Tricky

Normal equations for the least squares problem

$$\mathbb{X}^T T = \mathbb{X}^T \mathbb{X} w$$

Multiplying both sides by  $(\mathbb{X}^T \mathbb{X})^{-1}$

$$(\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T = w$$

Takeaway

- Regularized Least Squares
- When Normal Equation Gets Tricky (棘手)

$$w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T$$

Assuming  $\mathbb{X}^T \mathbb{X}$  is invertible

# Matrix Notation

$$\begin{cases} y(\vec{x}_1, \vec{w}) = \vec{x}_1^T \vec{w} & \text{where } \vec{x}_1^T = [1, x_{11}, x_{12}, \dots, x_{1D}] \\ y(\vec{x}_2, \vec{w}) = \vec{x}_2^T \vec{w} & \text{where } \vec{x}_2^T = [1, x_{21}, x_{22}, \dots, x_{2D}] \\ \dots \\ y(\vec{x}_N, \vec{w}) = \vec{x}_N^T \vec{w} & \text{where } \vec{x}_N^T = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \end{cases}$$

$$\begin{cases} y(\vec{x}_1, \vec{w}) = [1, x_{11}, x_{12}, \dots, x_{1D}] \vec{w} \\ y(\vec{x}_2, \vec{w}) = [1, x_{21}, x_{22}, \dots, x_{2D}] \vec{w} \\ \dots \\ y(\vec{x}_N, \vec{w}) = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \vec{w} \end{cases}$$

$$\mathbb{X} = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\} \in \mathbb{R}^{N \times (D+1)}$$

Design matrix

$$\mathbb{Y} = \{y(\vec{x}_1, \vec{w}), y(\vec{x}_2, \vec{w}), \dots, y(\vec{x}_N, \vec{w})\} \in \mathbb{R}^N$$

$$\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$$

$$\in \mathbb{R}^{N \times (D+1)}$$

$$\in \mathbb{R}^{D+1}$$

# SVD Recap

## 一、SVD 的标准定义

对任意矩阵：

$$X \in \mathbb{R}^{N \times d}$$

都可以分解为：

$$X = U \Sigma V^T$$

其中：

- $U \in \mathbb{R}^{N \times N}$ ：左奇异向量（正交矩阵）
- $V \in \mathbb{R}^{d \times d}$ ：右奇异向量（正交矩阵）
- $\Sigma \in \mathbb{R}^{N \times d}$ ：对角矩阵

### ✓ 核心结构

$$\Sigma = \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \end{bmatrix}$$

满足：

$$\sigma_1 \geq \sigma_2 \geq \dots \geq 0$$

## 二、最本质的一句话



SVD 把一个复杂线性变换分解为：旋转 → 拉伸 → 再旋转

## 三、几何直觉（非常关键）

考虑一个向量  $x$ ，经过  $X$  变换：

$$Xx = U \Sigma V^T x$$

可以分三步理解：

### Step 1: $V^T$

✎ 把输入空间“旋转”

- 改变坐标系
- 找到最重要的方向

# SVD Recap

## Step 2: $\Sigma$

👉 在每个方向上“缩放”

- $\sigma_i$ : 拉伸/压缩程度
  - 小  $\sigma \rightarrow$  被压扁
  - $\sigma=0 \rightarrow$  直接压没 (信息丢失)
- 

## Step 3: $U$

👉 再旋转到输出空间

# When Normal Equation Gets Tricky

- When is  $\mathbb{X}^T \mathbb{X}$  invertible?

$$\text{rank}(X) = D + 1 \iff \det(X^T X) \neq 0 \iff \sigma_i > 0, \forall i$$

- Full column rank:  $\text{rank}(\mathbb{X}) = D + 1$
- No perfect collinearity (共线性):  $\det(\mathbb{X}^T \mathbb{X}) \neq 0$
- All singular values (奇异值) be non-zero:  $\sigma_1, \sigma_2, \dots, \sigma_{D+1} > 0$

## ► Step 1: rank $\Leftrightarrow$ 奇异值

$$\text{rank}(X) = \#\{\sigma_i > 0\}$$

满足条件  $\sigma_i > 0$  的元素个数

👉 因此:

$$\text{rank}(X) = D + 1 \iff \sigma_i > 0 \forall i$$

## ► Step 2: 奇异值 $\Leftrightarrow$ 行列式

$$\det(X^T X) = \prod_{i=1}^{D+1} \sigma_i^2$$

👉 因此:

- 若存在  $\sigma_i = 0 \rightarrow \det = 0$
- 若全部  $\sigma_i > 0 \rightarrow \det \neq 0$

$$\det(X^T X) \neq 0 \iff \sigma_i > 0, \forall i$$

# When Normal Equation Gets Tricky

- Illconditioning (numerical instability)
  - Even if  $\det(\mathbf{X}^T \mathbf{X}) \neq 0$ 
    - **Condition number**  $\kappa = \frac{\sigma_{max}}{\sigma_{min}} \gg \text{threshold}$  (empirical value, such as  $10^8$ )
      - small noise in the data creates huge weight swing

在求解  $\mathbf{X}\mathbf{w} = \mathbf{y}$  时，误差放大的严谨公式是：

$$\frac{\|\delta \mathbf{w}\|}{\|\mathbf{w}\|} \leq \kappa \cdot \frac{\|\delta \mathbf{y}\|}{\|\mathbf{y}\|}$$

- 左边是输出的**相对误差**。
- 右边是输入的**相对误差**。  
这里的  $\delta \mathbf{y}$  可以是任何误差，比如传感器测量的随机噪声、数据采集的偏差等。

# Fix #1: Ridge Trick

Stabilize with  $(\mathbb{X}^T \mathbb{X} + \lambda I) \mathbf{w} = \mathbb{X}^T \mathbf{t}$

- $\lambda > 0$  boosts all singular values  $\rightarrow$  lower  $\kappa$ .
- Closed-form solution:  $\mathbf{w}^* = (\mathbb{X}^T \mathbb{X} + \lambda I)^{-1} \mathbb{X}^T \mathbf{t}$

- $\lambda$  (正则化参数): 一个大于 0 的超参数。
- $I$  (单位矩阵): 与  $\mathbb{X}^T \mathbb{X}$  同维度的单位矩阵。

## 2. 为什么这种方法有效?

这种“技巧”之所以被称为 **Ridge Trick** (岭技巧), 是因为它在数学和数值上带来了显著改善:

- **提升奇异值 (Boosting Singular Values):**

通过加上  $\lambda I$ , 原矩阵的所有奇异值 (或特征值) 都会增加  $\lambda$ 。

- 旧奇异值:  $\sigma_i$
- 新奇异值:  $\sigma_i + \lambda$

- **大幅降低条件数 ( $\kappa$ ):**

原本的条件数是  $\frac{\sigma_{max}}{\sigma_{min}}$ 。加上  $\lambda$  后, 新的条件数为:

$$\kappa_{new} = \frac{\sigma_{max} + \lambda}{\sigma_{min} + \lambda}$$

即使原本的  $\sigma_{min}$  趋近于 0, 分母现在也有了  $\lambda$  作为“保底”。这使得条件数大幅减小, 从而抑制了舍入误差的放大。

## 3. 解析解 (Closed-form Solution)

通过上述改进, 我们可以得到岭回归的最优权重公式 (解析解):

$$\mathbf{w}^* = (\mathbb{X}^T \mathbb{X} + \lambda I)^{-1} \mathbb{X}^T \mathbf{t}$$

### 💡 数值直觉:

只要  $\lambda > 0$ , 矩阵  $(\mathbb{X}^T \mathbb{X} + \lambda I)$  就一定是正定且可逆的。即使特征之间存在严重的共线性, 这个公式也能给出一个稳定的、唯一的解。



# Illconditioning Example

With the condition factor that is large, due to very small eigenvalues resulting from very similar features, the weights become unstable.

Try changing the noise and you will see substantial swings in weights.

```
N, D = 500, 2 # one bias + 2 features
```

```
X = np.ones((N, D + 1)) # first column = bias
```

```
x1 = np.random.randn(N)
```

```
x2 = x1 + 1e-4 * np.random.randn(N) # almost identical feature
```

```
X[:, 1] = x1
```

```
X[:, 2] = x2
```

Condition  $(\mathbf{X}^T \mathbf{X}) =$   
420140223.27925146

```
t = 3 + 2*x1 - 1*x2 + 0.1*np.random.randn(N) # targets
```

```
XtX = X.T @ X
```

```
Xty = X.T @ t
```

```
w_ne = np.linalg.solve(XtX, Xty)  $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$ 
```

```
lam = 1e-2
```

```
w_ridge = np.linalg.solve(XtX + lam*np.eye(D+1), Xty)  $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} \mathbf{X}^T \mathbf{t}$ 
```

| 方法                           | 预测权重结果 (Partial)                     | 状态评价                       |
|------------------------------|--------------------------------------|----------------------------|
| 真实值 (Ground Truth)           | [3.0, 2.0, -1.0]                     | 理想状态                       |
| 正规方程 ( <code>w_ne</code> )   | [3.0, <b>12.87</b> , <b>-11.87</b> ] | 崩溃: 权重发生了剧烈摆动 (Huge Swing) |
| 岭回归 ( <code>w_ridge</code> ) | [3.0, 0.50, 0.49]                    | 稳定: 虽然有偏差, 但数值极度稳健         |

# Fix #2: Moore–Penrose Pseudo-Inverse (摩尔-彭若斯伪逆)

- Singular Value Decomposition (SVD) recipe:
  - $X = U\Sigma V^T$  where  $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r)$  with  $r = \text{rank}(X)$ .
  - Every real matrix can be factorized this way;  $U$  and  $V$  have orthogonal columns ( $U^T U = V^T V = I$ ),  $\Sigma$  holds the “stretch factors”.

- Moore–Penrose Pseudo-inverse:

$$\Sigma_{ii}^+ = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i > \varepsilon \sigma_{\max} \\ 0 & \text{otherwise} \end{cases}$$

- We safely “flip” only the non-tiny singular values.

$$X^+ = V\Sigma^+ U^T$$

- Penrose conditions:  $XX^+X = X$ ,  $X^+XX^+ = X^+$ ,  $(XX^+)^T = XX^+$ ,  $(X^+X)^T = X^+X$ . These make  $X^+$  unique.

# Fix #2: Moore–Penrose Pseudo-Inverse

How can Penrose Pseudo-Inverse help us with solving normal equation?

- Moore–Penrose Pseudo-inverse:

$$\Sigma_{ii}^+ = \begin{cases} \frac{1}{\sigma_i} & \text{if } \sigma_i > \varepsilon \sigma_{max} \\ 0 & \text{otherwise} \end{cases}$$

- We safely “flip” only the non-tiny singular values.

Closed-form solution for least squares:

$$\mathbf{w} = \mathbf{X}^+ \mathbf{t}$$

This works no matter whether  $\mathbf{X}$  is tall ( $N > D$ ), square, or wide ( $N < D$ ). Also regardless of singular values.

# Fix 3: Sequential Learning

- In practice, we use sequential algorithms, also known as online algorithms, in which the data points are considered one at a time and the model parameters updated after each such presentation.
- **Algorithm:** Stochastic gradient descent, also known as sequential gradient descent:
  - After presentation of datapoint  $n$ , the gradient descent algorithm updates parameters using  $w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E_n$

## Lecture 6

# Linear Regression (2)

**Reference Book Chapters:** Chapter 1.2, Chapter 4.[1.4-1.6]